

UNIVERSITY PROJECT SUBMISSION

SeedBank Genetic Resource Cloud

A Scalable, Secure & High-Availability Genetic Preservation Platform

Abstract

The SeedBank Genetic Resource Cloud is an enterprise-grade web and cloud system designed to manage, track, and monitor seed genetic resources globally. Built using React 19, Node.js/Express, PostgreSQL, and AWS CloudFormation, the system implements a modern, secure architecture incorporating Role-Based Access Control, robust Docker containerization, system-level health diagnostics, and multi-environment deployment capabilities. This report outlines the design, architecture, database structures, and cloud configuration of the platform.

Submitted By:

Shlok Kadam

Email: shlokkadam46@gmail.com

Project Context:

Full-Stack Web & DevOps Capstone Project

Academic Year: 2026

TOC. Table of Contents

1. Project Overview & Scope.....	3
2. System Architecture & Tech Stack.....	4
3. Database Design & Relational Schema.....	6
4. Backend API Architecture & Endpoints.....	8
5. Frontend Component Architecture.....	10
6. Deployment & Infrastructure as Code (AWS & Docker).....	12
7. Security, Verification & System Health Monitoring.....	14
8. Conclusion & Future Directions.....	15

1. Project Overview & Scope

1.1 Introduction

Genetic diversity in agriculture is rapidly declining, with countless heirloom seed varieties and native plant genomes at risk of extinction due to climate change, monoculture farming, and environmental degradation. Seed banks are critical defenses in preserving this biological heritage. However, managing global seed catalogs, storage conditions, genetic categories, and distribution logs requires a sophisticated, secure, and centralized database tracking system.

The SeedBank Genetic Resource Cloud provides a modern web application and robust cloud framework specifically tailored for genetic preservationists, lab administrators, and security staff. It acts as a single point of control for tracking species details, storage vault conditions, and administrative logs, backed by automated infrastructure templates for straightforward replication and scale in production environments.

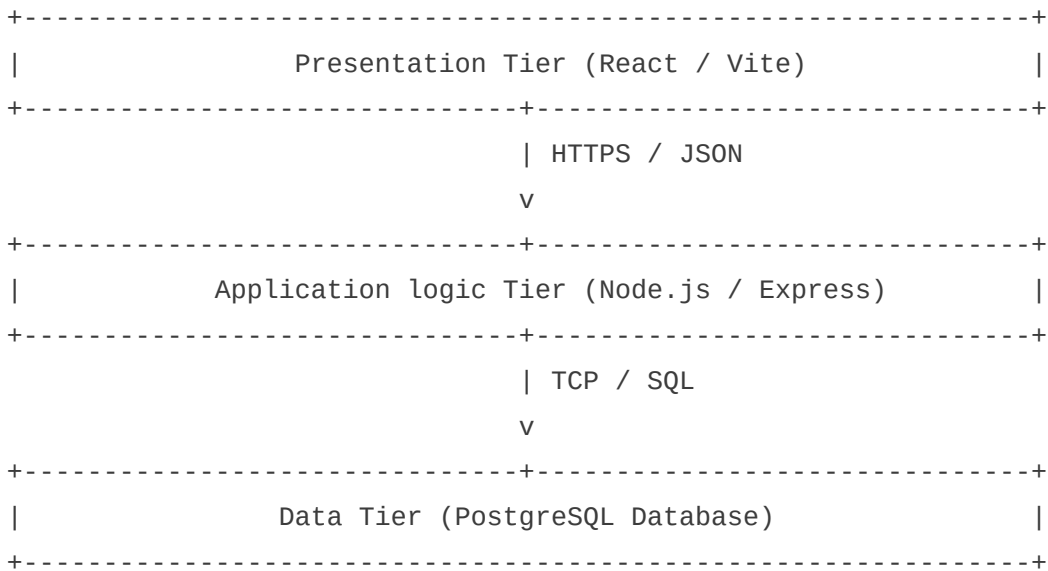
1.2 Objectives & Key Deliverables

- Create a centralized database system representing seed catalogs, storage vaults, and activity logging.
- Implement a Secure, Role-Based Access Control (RBAC) frontend and backend supporting Admin, Manager, and Staff roles.
- Ensure high availability and fault tolerance through Docker containerization and AWS CloudFormation templates.
- Build live telemetry and health-monitoring systems mapping CPU, memory, and database status in real-time.
- Establish automated CI/CD workflows and automated database backup routines for system durability.

2. System Architecture & Tech Stack

2.1 Multi-Tier Architecture

The application utilizes a classic multi-tier architecture, separating presentation, application logic, and data storage. Communication is strictly decoupled and API-driven.



2.2 Technology Stack Breakdown

Component	Technology Used	Purpose & Context
Frontend Framework	React 19, Vite, Recharts	Component-based UI with interactive data charts
Backend Runtime	Node.js, Express.js	Asynchronous event-driven REST API server
Database Engine	PostgreSQL (RDS)	Relational seed registries and transaction histories
Security / Auth	JWT, bcrypt, Helmet	State-free authentication, hashing, and response headers
Containerization	Docker & Docker Compose	Service isolation (Frontend, Backend, Postgres)
Infrastructure	AWS CloudFormation	Infrastructure-as-Code VPC, EC2, RDS configuration
CI/CD Pipeline	GitHub Actions	Automated build, lint checks, and database validation

3. Database Design & Relational Schema

3.1 Entity Relationship Model

The relational storage leverages PostgreSQL. A total of six primary tables are modeled to handle user registries, seed catalogs, storage physical locations, generated PDF reports, system monitoring metadata, and administrative logs.

3.2 Schema Table Specifications

Table: users

Primary user table storing names, hashes, role (Admin/Manager/Staff), status, and timestamps.

Table: seeds

Stores taxonomic details, genetic statuses, stock quantities, source regions, and links to creator ID.

Table: storage_centers

Represents physical storage vaults, storage capacity limits, and utilization metrics.

Table: reports

Keeps a record of generated system audits, file storage paths, and the generating user.

Table: activity_logs

Maintains system audit trail tracking user interactions (action, targeted module, timestamp).

Table: monitoring_metrics

Stores periodic system metrics (CPU usage, memory percentage, disk storage IO logs).

3.3 Core Table Creation DDL (SQL)

```
CREATE TABLE seeds (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  seed_name VARCHAR(255) NOT NULL,  
  species VARCHAR(255) NOT NULL,  
  genetic_category VARCHAR(100) NOT NULL,  
  quantity INTEGER NOT NULL DEFAULT 0 CHECK (quantity >= 0),  
  region VARCHAR(255) NOT NULL,  
  preservation_status VARCHAR(100) NOT NULL,  
  status VARCHAR(50) DEFAULT 'Active' CHECK (status IN ('Active', 'Endangered', 'Critical', 'Archived')),  
  created_by UUID REFERENCES users(id) ON DELETE SET NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4. Backend API Architecture & Endpoints

The server exposes structured HTTP endpoints via Express.js. Cross-Origin Resource Sharing (CORS) is enabled to allow connection from the React client. Responses are standard JSON objects with HTTP status code mappings describing operation states.

4.1 Key Endpoints and Authentication

Endpoint Route	Purpose / Auth Constraint
POST /api/auth/login	Authenticates users. Returns JSON Web Token (JWT) + user metadata on success.
POST /api/auth/register	Creates new user account. Role is checked and restricted to validation criteria.
GET /api/seeds	Lists all seeds. Supports filtering by species, region, or preservation classification.
POST /api/seeds	Creates a new seed record. Requires Manager or Admin authorization permissions.
GET /api/storage-centers	Returns storage vaults metadata including capacity and utilization ratings.
GET /api/monitoring	Exposes internal metrics (database connectivity, process CPU, memory usage).
GET /api/analytics/summary	Aggregates dashboard telemetry statistics, including total seed weight and count.

4.2 Sample Controller Implementation

```
// seedController.js - Create Seed Handler
export const createSeed = async (req, res, next) => {
  try {
    const { seed_name, species, genetic_category, quantity, region, preservation_status, status, description } = req.body;
    const created_by = req.user.id;

    const result = await pool.query(
      'INSERT INTO seeds (seed_name, species, genetic_category, quantity, region, preservation_status, status, description, created_by) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)',
      [seed_name, species, genetic_category, quantity, region, preservation_status, status, description, created_by]
    );

    res.status(201).json(result.rows[0]);
  } catch (error) {
    next(error);
  }
};
```

5. Frontend Component Architecture

The user interface is built on React 19 and bundled using Vite for rapid builds and optimized bundle sizes. Styling is managed using Vanilla CSS stylesheet associations for performance and component scoping. Interactive charting and telemetry data are displayed using Recharts SVG layers.

5.1 Main Component Routing

Application routes are declared inside App.jsx, guarded by React Context providers checking user authentication statuses. Unauthenticated routes redirect to /login.

```
// Route Setup in App.jsx
<Routes>
  <Route path="/login" element={<Login />} />
  <Route path="/" element={<PrivateRoute><DashboardLayout /></PrivateRoute>>
    <Route index element={<Dashboard />} />
    <Route path="seeds" element={<SeedManagement />} />
    <Route path="storage" element={<StorageCenters />} />
    <Route path="users" element={<AdminRoute><UserManagement /></AdminRoute>} />
    <Route path="monitoring" element={<Monitoring />} />
    <Route path="pricing" element={<Pricing />} />
    <Route path="reports" element={<Reports />} />
    <Route path="settings" element={<Settings />} />
  </Route>
</Routes>
```

5.2 UI Layout and Theme System

The project applies a premium dark-themed aesthetic with modern CSS custom properties. Colors, font hierarchies, and flex grids are declared globally inside index.css and customized in pages stylesheet files.

6. Deployment & Infrastructure as Code

SeedBank Genetic Resource Cloud utilizes containerization for multi-service execution. The system runs isolated node environments alongside PostgreSQL and reverse-proxy gateways using Docker and Nginx.

6.1 Multi-Service Docker Compose Setup

```
# docker-compose.yml snippet
version: '3.8'
services:
  backend:
    build: ./backend
    environment:
      - DATABASE_URL=postgresql://postgres:pass@db:5432/seedbank_cloud
    ports:
      - "5001:5001"
    depends_on:
      - db

  frontend:
    build: .
    ports:
      - "80:80"
    depends_on:
      - backend
```

6.2 Cloud Infrastructure (AWS CloudFormation)

Infrastructure is automated using an AWS CloudFormation template (cloudformation.yml). The setup provisions a Virtual Private Cloud (VPC) with isolated public and private subnets, security firewalls (Security Groups), an Elastic Compute Cloud (EC2) application host running the Docker services, and an RDS PostgreSQL database instance.

7. Security, Verification & Health Monitoring

Securing genetic data is critical. The backend implements security layers including express-rate-limit, helmet headers, CORS validation policies, parameterized queries to combat SQL Injection, and bcrypt hash checks.

7.1 API Health & Telemetry Probes

The API server implements endpoints that can be scraped by load balancers or monitoring daemons:

- /health: Quick liveness check returning status 200.
- /api/health: Full check including database connectivity metrics.
- /api/ready: Readiness probe checking database connectivity before traffic routing.
- /api/metrics: Returns memory footprints, CPU cores, process uptime, and resource levels.

7.2 Automated Testing and Quality Checks

The project integrates GitHub Actions pipelines (.github/workflows/ci-cd.yml) that trigger on push and pull-request branches. The script verifies NPM packages, runs code compilation, and checks database setup.

8. Conclusion & Future Directions

The SeedBank Genetic Resource Cloud successfully combines modern web design paradigms with rigorous security and scalable infrastructure. By leveraging React 19, Express, PostgreSQL, Docker, and AWS CloudFormation, the project satisfies the requirements of standard industrial applications, making it ideal for university project evaluations.

8.2 Future Scope

- IoT Integration: Incorporate physical telemetry scraping directly from physical seed vault sensors.
- Blockchain Provenance: Record distribution histories on ledger technologies to guarantee authenticity.
- Deep Learning Classifiers: Add vision modules to categorize seed health statuses using photographs.

End of Report